

Contact Finder Pipeline — Build Spec

Oct 1, 2026 · @Grant

Overview

Contact Finder turns any pasted text about people and companies into an outreach-ready contact list with best-guess email addresses, in one click. It replaces the manual loop of copy-paste → ChatGPT parse → Google the email format → paste snippets back → build variants → CSV.

The tool does three things: extract structured contacts from messy text, discover each company's email format from public search results, and generate up to 3 ranked email candidates per person. Output is a table that exports to CSV and, later, pushes straight into a CRM.

v1 is a single-page browser app with one tiny backend function. The same stage functions become MCP tools for Claude Desktop or a ChatGPT app in v2, so nothing is rewritten.

Who it's for: anyone doing outbound who already has names or target companies but not emails — founders, recruiters, BD, agency owners.

Input modes

The user pastes anything into one box. A classifier stage decides what it got and routes it; the user never pre-sorts companies from names.

Input looks like	Examples	Route
People + company	LinkedIn search results, team/leadership page copy, conference attendee list, email signature dump	People-first: extract → resolve domain → pattern → candidates
Companies only	List of company names, a market map, a CSV column of logos' names	Company-first: find website → find team page → extract people → people-first path
URLs	Company homepage, /team or /about page, a directory listing page	Fetch page text → classify as above

Input looks like	Examples	Route
Mixed	Notes like "Acme (acme.com) — Jane Doe, VP Sales; Beta Corp — need CFO"	Split per company, route each part

Rules:

- The classifier returns `{mode, companies[], people[], urls[]}` and the UI shows what it found before running the expensive stages, so the user can fix a misparse early.
- A company-first run asks for a target role filter (e.g. "VP Sales, Head of Marketing") before extracting people, otherwise a 300-person team page produces 300 rows.
- URLs are fetched server-side through the same Claude call (web fetch tool) — no scraper code, no headless browser.
- LinkedIn is never crawled by the app. The user copies from LinkedIn into the box themselves, same as today; the app only parses pasted text.

Pipeline stages

Each stage is a pure function `(input, ctx) → output` with its own prompt, tool schema, and unit test. Stages run per contact but share a per-company cache so 40 people at one company trigger one domain lookup and one pattern search.

1. **Classify & extract** — Claude turns the pasted text into Contact and Company rows (first, last, title, company, LinkedIn URL, website if present). Dedupes on `first+last+company`. Strips credentials, pronouns, emoji, and "1st/2nd" LinkedIn badges. No web calls.
2. **Resolve domain** — per unique company: one web search for `<company name> official site`, Claude picks the canonical domain, rejects aggregators (LinkedIn, Crunchbase, Wikipedia, Bloomberg). Stores `domain`, `confidence`, `source_url`. Skipped when the input already carried a website.
3. **Discover pattern** — per domain: 2–3 searches, e.g. `"@acme.com" email format`, `acme.com email`, `acme.com employee email`. Claude reads result snippets (RocketReach, SignalHire, Hunter, LeadIQ pages expose the format in their preview text) and returns up to 3 patterns with confidence and the URL each came from. Also harvests any literal emails seen in snippets as evidence. No pattern found → fall back to statistical defaults: `{first}.{last}` (~45%), `{first}` (~15%), `{f}{last}` (~15%).

4. **Generate candidates** — apply the top patterns to each name, max 3 per contact, ranked by pattern confidence. Normalizes: lowercase, strip accents (José → jose), drop middle names and suffixes (Jr., III), hyphenated last names tried joined and first-part-only, common nickname map (Bob→Robert) only as a 4th candidate if slots remain.
5. **Verify (optional, pluggable)** — v1: MX record lookup per domain (free; catches dead domains and typos). v2: a verifier adapter interface with one implementation (NeverBounce / ZeroBounce / Hunter) that marks each candidate `valid / risky / invalid / catch-all` and promotes the valid one to `primary_email`.
6. **Export** — results table → CSV (one row per contact, candidates in `email_1..3`, plus `pattern`, `pattern_confidence`, `pattern_source_url`, `domain_source_url`, `verified_status`). v2: CRM push adapter (HubSpot first; same adapter shape as the verifier).

Stage contract: every stage emits `{ok, data, confidence, sources[], tokens_used}` so the UI can show provenance and cost per row and the pipeline can retry a single failed stage without rerunning the rest.

Data model

Four types, all plain JSON, held in browser state (and localStorage) in v1; the same shapes become Supabase tables when multi-device or team use is needed.

```
type Company = {
  id: string;           // slug of normalized name
  name: string;
  domain?: string;      // "acme.com"
  domain_confidence?: number; // 0-1
  domain_source_url?: string;
  patterns: Pattern[];  // ranked, max 3
  mx_ok?: boolean;
  fetched_at?: string;  // ISO date, for cache expiry (30 days)
};

type Pattern = {
  template: string;      // "{first}.{last}" | "{f}{last}" | "{first}" | "{first}_{last}" | "{first}{last}" | "{last}.{first}" | "{f}.{last}" | "{last}"
  confidence: number;    // 0-1
  source_url?: string;   // page the format was read from; absent = statistical default
}
```

```

    evidence?: string[];    // literal emails seen in snippets
  };

  type Contact = {
    id: string;
    first: string;
    last: string;
    title?: string;
    company_id: string;
    linkedin_url?: string;
    raw_source: string;    // the pasted chunk this came from, for debugging
    candidates: Candidate[]; // max 3
    primary_email?: string; // set when a candidate verifies, else
    candidates[0]
      status: "pending" | "ok" | "no_domain" | "no_pattern" | "error";
  };

  type Candidate = {
    email: string;
    pattern: string;
    rank: 1 | 2 | 3;
    verify_status?: "valid" | "risky" | "invalid" | "catch_all" | "unverified";
  };

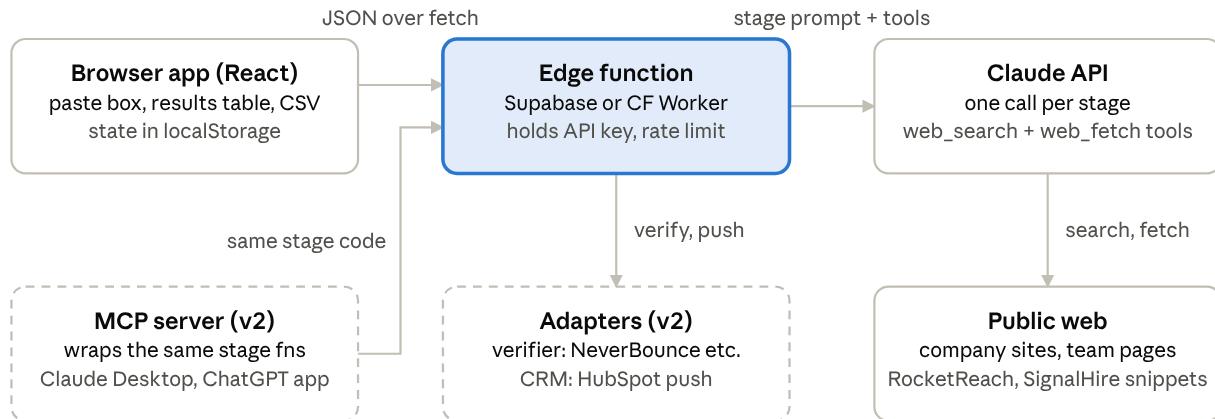
```

CSV export flattens Contact + Company into one row: first, last, title, company, domain, email_1, email_2, email_3, pattern, pattern_confidence, pattern_source_url, linkedin_url, verify_status, opt_out (opt_out is an empty column the CRM can fill).

Architecture

Three live parts in v1: a static React app, one edge function that holds the Anthropic key, and the Claude API doing both the LLM work and the web searching. No scraper, no database, no search-API vendor.

One edge function fronts the Claude API; the browser holds the data



v1 architecture · 3 live parts, 2 v2 parts

The browser owns all data and calls the edge function once per stage; the edge function forwards the stage prompt to Claude with the web_search and web_fetch tools enabled, so search snippets and page text come back inside the same response.

Key choices:

- **Claude API server-side web search** replaces Serper/Brave/Tavily. One vendor, one key, and the model reads snippets in the same call it reasons over them. Web search is billed per search on top of tokens; budget roughly 3–4 searches per new company (1 domain + 2–3 pattern), none per additional person at that company.
- **Model**: Sonnet-class for extract and pattern stages (cheap, fast, good enough for parsing); Haiku-class for the classifier. Make the model id a config constant.
- **Edge function responsibilities only**: inject key, enforce a per-IP or per-session rate limit, log token/search counts, optionally cache `domain → patterns` in Supabase KV for 30 days so repeat runs on the same companies cost nothing. Nothing else lives server-side.
- **Hosting**: static app on Vercel/Netlify/Cloudflare Pages; edge function on Supabase (already set up) or a Cloudflare Worker. Either way, under 100 lines of server code.
- **Concurrency**: run companies in parallel (limit 5 at a time), contacts within a company sequentially after the pattern is known. Show per-row progress.

Prompts and tool schemas

Every stage forces structured output with a single `tool_use` schema and `tool_choice`: `{type: "tool", name: ...}` so there is no JSON-fence parsing. Keep prompts in `/prompts/*.md` so they can be tuned without touching code.

Stage 1 — `classify_and_extract` (no web tools)

System: You extract people and companies from messy pasted text (LinkedIn results, team pages, notes, signatures). Return only what is present; never invent titles or companies. Strip connection badges ("1st", "2nd", "• 3rd+"), pronouns, credentials after commas (MBA, PhD), and emoji. If a line has a name but no company, attach the nearest company heading above it. Dedupe on first+last+company.

Tool: `extract_contacts` → `{mode: "people"|"companies"|"urls"|"mixed", companies: [{name, website?}], people: [{first, last, title?, company, linkedin_url?}], urls: string[], notes: string}`

Stage 2 — `resolve_domain` (web_search enabled, max 2 searches)

System: Find the official website domain for the company. Prefer the company's own site over LinkedIn, Crunchbase, Wikipedia, Bloomberg, ZoomInfo. If the company name is ambiguous (e.g. "Apex"), use the title/industry hint. Return the bare domain without www. Set confidence < 0.5 if you had to guess between candidates.

Tool: `report_domain` → `{domain: string|null, confidence: number, source_url: string|null, alternatives: string[]}`

Stage 3 — `discover_pattern` (web_search enabled, max 3 searches)

System: Discover the email address format used at `{domain}`. Search for the domain's email format and read the result snippets. Pages from RocketReach, SignalHire, Hunter, LeadIQ, ContactOut, and Apollo often state it in the snippet (e.g. "most common pattern is first.last"). Also collect any literal `@{domain}` addresses you see and infer the pattern from them. Return up to 3 patterns, ranked. Use only these template tokens: `{first}` `{last}` `{f}` `{l}` and separators `.` `_` `-` or none. If you find nothing, return an empty list; do not guess.

Tool: `report_patterns` → `{patterns: [{template, confidence, source_url?, evidence?: string[]}]}`

Stage 4 — `generate_candidates` (pure code, no LLM)

Deterministic function. Normalization order: lowercase → NFD strip diacritics → remove apostrophes and periods → split on space/hyphen → first token = first, last token = last → apply template → max 3, dedupe. Unit-test with: José Álvarez-Ruiz, Mary Anne O'Brien, Dr. Li Wei PhD, Bob Smith Jr..

Stage 5 — verify (adapter interface)

```
interface Verifier {
  name: string;
  verify(emails: string[]): Promise<Record<string,
    "valid"|"risky"|"invalid"|"catch_all"|"unverified">>;
}
```

v1 ships `MxVerifier` (DNS MX lookup from the edge function; marks whole domain). `NeverBounceVerifier` / `ZeroBounceVerifier` are drop-ins behind the same interface.

Company-first add-on — find_people (web_search + web_fetch, max 3 calls)

System: Find the team or leadership page for `{domain}` and extract people whose titles match `{role_filter}`. Fetch the page; do not rely on snippets alone. Return the same shape as `extract_contacts`.

Decision layer (optional, v2)

The pipeline's choose/classify/score steps go behind one `DecisionProvider` interface so a fast typed-decision model such as [Jev \(TypeSafe AI\)](#) can replace Claude for those calls without touching the rest. Jev returns typed outputs with calibrated probabilities in 70–500 ms and cannot generate text, so it fits decisions, not extraction. It is early access as of September 2026; v1 ships with the Claude implementation.

```
interface DecisionProvider {
  name: string;
  classify(labels: string[], context: string): Promise<Record<string,
    number>>; // label → probability
  choose(options: string[], context: string): Promise<{ index: number; probs:
    number[] }>;
  score(context: string, question: string): Promise<number>;
  // 0-1
}
```

Implementations: `ClaudeDecisions` (tool-use with an enum schema; v1) and `JevDecisions` (v2). Everything generative — stage 1 extraction, company-first page parsing, running web searches — stays on Claude regardless.

Use	Call	What it adds
Instant Parse preview	<code>classify</code> each pasted line: name / title / company / location / badge / noise	Preview appears in under a second; for regular pastes (LinkedIn results, team pages) code assembles rows from labels and skips the Claude extraction call
Entity resolution	<code>score</code> on pairs: "same company?" / "same person?"	Reliable dedup when merging lists from several sources; <code>Apex Inc</code> vs <code>Apex Technologies</code>
ICP fit scoring	<code>score</code> contact + company against the user's ideal-customer text	Table sorts best leads first; export gains an <code>icp_score</code> column
Snippet pre-filter	<code>score</code> each search snippet for "states an email format for this domain"	Only the top 5 snippets go to Claude; fewer tokens, faster stage 3
Pattern choice	<code>choose</code> among the 8 templates given the surviving snippets	Calibrated <code>pattern_confidence</code> instead of a self-reported number
Domain pick	<code>choose</code> among search results for the official site	Low-probability picks flag the row for review
Rescue gate	score after the domain and pattern stages: "is this result right given the evidence?"; <code>classify</code> each company as easy / ambiguous / subsidiary before searching; score "good enough?" after each rescue tool call	Catches rows that are ok but wrong, routes hard companies straight to the agent, ends rescue loops early; falls back to the status-only rule when Jev is unavailable
Title normalization	<code>classify</code> title into seniority × function taxonomy	Clean CRM segmentation fields
Row sanity judge	<code>score</code> "is this a real person, not a job posting or company?"	Catches extraction errors before searches are spent

Trade-off: once Jev makes the decision, the search snippets must be fetched first by a search API (Serper, Brave) instead of inside the Claude call, so this layer adds a second vendor. Worth it at thousands of rows or when the live preview matters; not needed for a 50-contact run.

Orchestration

A fixed pipeline runs every row; an agent is called only for rows that fail. Not agent-first: the happy path is identical for 90 % of rows, and a loop does it with predictable cost, parallelism, and per-row progress that an agent cannot show.

Mode	Who decides the next step	When it runs	Budget
Pipeline	Code (stages 1–6 in order)	Every row	≈ 4 searches per new company, 0 per extra person
Rescue agent	Claude tool-use loop with the stage functions + web search as tools	Rows ending <code>no_domain</code> , <code>no_pattern</code> , or <code>error</code> ; ambiguous companies	Max 8 tool calls per row, then give up with a reason
Chat / MCP	Host model (Claude Desktop, ChatGPT)	v2 conversational use	Host’s own limits

Rescue agent contract: input = the failed Contact + Company + what the pipeline already tried; tools = `find_domain`, `find_email_pattern`, `find_people`, `web_search`, `web_fetch`; output = either a repaired row with `source_urls` or `{gave_up: true, reason}`. Typical wins: parent-company domains, renamed companies, patterns read from a fetched team page rather than snippets. Trigger: row status in v1; in v2 the Decision layer’s rescue gate (calibrated probability below a tuned threshold, about 0.6), so rows that are ok but silently wrong are caught too.

Not in v1: Claude Agent SDK subagents or OpenAI multi-agent sessions. Both buy parallel researcher agents that a 50-contact run does not need, and both require a Node/Python runtime rather than an edge function. Revisit the Agent SDK for the MCP server if a per-company researcher fan-out becomes worth it.

UI spec

One screen, three zones top to bottom. No login, no settings page in v1.

1. Input zone

- One large textarea with placeholder: "Paste names, companies, team pages, LinkedIn results, URLs — anything."
- Optional single-line field: "Target roles (company-first only)", e.g. VP Sales, Head of Growth.
- Buttons: **Parse** (runs stage 1 only, fast, no searches) and **Run pipeline**.
- After Parse: a compact preview — "Found 14 people at 6 companies" with an editable mini-table so the user fixes a wrong company before paying for searches.

2. Results table (one row per contact, updates live as stages finish)

Column	Notes
Name, Title	editable inline
Company → domain	domain shows a link icon to <code>domain_source_url</code> ; red if confidence < 0.5
Pattern	e.g. <code>first.last</code> with a confidence pill and a link to the page it came from; grey "default" badge when statistical fallback
Email 1 / 2 / 3	copy-on-click; verified one gets a green check
Status	pending / ok / no_domain / no_pattern / error, with a Retry on failed rows

Filters above the table: show only `ok` , hide rows without a found pattern, group by company.

3. Export bar (sticky bottom)

- **Copy as table** (TSV to clipboard, pastes into Sheets)
- **Download CSV** (schema in Data model)
- **Push to CRM** — disabled in v1 with tooltip "coming soon"
- Running cost counter: `searches: 23 · tokens: 41k · ≈ $0.38`

States and edge cases

- Paste > 500 contacts → warn and offer to run the first 100.
- Edge function 429 → pause and show "rate limited, resuming in 10 s".
- Zero people extracted → show what the classifier *did* find and suggest the company-first mode.
- Everything persists to localStorage keyed by session so a refresh doesn't lose a run; a **Clear** button wipes it.

MCP / ChatGPT wrapper (v2)

The stage functions are written once in `/core` with no React or edge-function imports, so v2 is a thin server that exposes them as tools and lets the host model (Claude Desktop, ChatGPT) do the orchestration conversationally.

MCP tool	Wraps	Input → output
<code>extract_contacts</code>	stage 1	<code>text</code> → <code>people[]</code> , <code>companies[]</code>
<code>find_domain</code>	stage 2	<code>company_name</code> , <code>hint?</code> → <code>domain</code> , <code>confidence</code> , <code>source</code>
<code>find_email_pattern</code>	stage 3	<code>domain</code> → <code>patterns[]</code>
<code>generate_emails</code>	stage 4	<code>first</code> , <code>last</code> , <code>domain</code> , <code>patterns[]</code> → <code>candidates[]</code>
<code>verify_emails</code>	stage 5	<code>emails[]</code> → <code>statuses</code>
<code>export_csv</code>	stage 6	<code>contacts[]</code> → CSV string
<code>run_pipeline</code>	all	<code>text</code> → full contact list (convenience)

Notes:

- Build it as a remote MCP server (HTTP/SSE) on the same edge platform, not stdio, so one deployment serves Claude Desktop, ChatGPT apps, and the web UI.
- Stages 2–3 already call the Claude API with web search internally; the MCP server keeps doing that rather than asking the host model to search, so results are identical across hosts.
- A ChatGPT app is the same server plus the app manifest; the results table can be returned as a widget later.

- **Auth:** a per-user API key issued by the edge function, passed as a bearer token, so usage is metered the same way as the web app.

Compliance and guardrails

The app parses text the user pastes and reads public search results; it never logs into or crawls LinkedIn or any site behind a login. That line keeps it in the same category as Hunter, RocketReach, and Apollo rather than a scraper.

- **No LinkedIn automation.** Do not add a browser extension that auto-scrolls LinkedIn or a fetcher that hits linkedin.com. Pasting is manual, by design.
- **Provenance on every guess.** Each pattern and domain carries its source URL; the export includes them so a reviewer can check where an address came from.
- **Mark guesses as guesses.** Unverified candidates export with `verify_status = unverified`; the UI never shows an unverified email as confirmed.
- **Opt-out column** in every export, and a visible note that the user is responsible for CAN-SPAM (US) / GDPR and PECR (EU, UK) compliance in their outreach — unsubscribe link, honest sender, suppression list.
- **Rate limits on the edge function** (per session and per day) so a leaked URL can't run up the API bill.
- **No storage of results server-side in v1.** Contacts live in the user's browser; the optional Supabase cache stores only `domain → patterns`, never names.
- **Respect robots and terms** on fetched team pages: the `web_fetch` tool is used for public pages only; skip any URL the classifier flags as a login-walled or paid data-vendor page.

Build order and acceptance tests

Six milestones, each shippable on its own; a coding agent can take them one at a time. Suggested repo layout: `/core` (stage functions + tests), `/web` (Vite + React app), `/edge` (one function), `/prompts` (markdown prompt files), `/fixtures` (sample pastes).

1. **Core + fixtures (no UI, no API).** Implement `generate_candidates` and the name normalizer; write 5 fixture pastes (LinkedIn search copy, a team page, a mixed note, a company-only list, a URL list). *Accept:* normalizer unit tests pass on the four tricky names in Prompts; fixtures load.

2. **Edge function + stage 1.** Deploy the proxy with key injection and a rate limit; wire `classify_and_extract`. *Accept:* each fixture returns the expected people/company count within ± 1 ; a 429 is returned after N calls per minute.
3. **Stages 2–3 with web search.** Add `resolve_domain` and `discover_pattern`; add the per-company cache. *Accept:* for a 10-company test list, ≥ 8 domains resolve correctly and ≥ 6 patterns come back with a source URL; total searches ≤ 40 .
4. **Web UI.** Paste $\rightarrow$ Parse preview $\rightarrow$ Run $\rightarrow$ live table $\rightarrow$ CSV. localStorage persistence. *Accept:* 30-contact fixture completes end to end in under 2 minutes; CSV opens in Sheets with the documented columns; refresh keeps the run.
5. **MX verify + polish.** `MxVerifier`, retry on failed rows, filters, cost counter. *Accept:* a dead domain fixture marks all its rows `no_domain`; cost counter matches API usage logs within 10 %.
6. **Company-first mode.** `find_people` with role filter. *Accept:* given 3 company names and "VP Sales", returns at least one matching person per company that has a public team page.

After 6: MCP server (one afternoon, since `/core` is host-agnostic), then the first CRM adapter.

Done-done checklist for v1:

- ☐ All six milestones' acceptance tests pass
- ☐ Prompts live in `/prompts` and can be edited without a rebuild
- ☐ README covers env vars, deploy steps, and the compliance notes
- ☐ A 50-contact real-world paste runs under \$1 in API cost

Open decisions before coding

Answer these in the README before milestone 2; defaults are given.

Decision	Default	Why it matters
Edge platform	Supabase Edge Function (already connected)	Picks the deploy command and where the cache lives
Search budget per company	4 searches max	Bounds cost; raise if pattern recall is low

Decision	Default	Why it matters
Verifier in v1	MX only	Real verification adds a paid vendor and signup
First CRM	HubSpot (free tier, simple contacts API)	Shapes the adapter interface
Max contacts per run	100	Keeps a run under ~2 min and ~\$2
Cache TTL for domain → patterns	30 days	Formats rarely change; stale cache is cheap to refresh
Nickname expansion (Bob → Robert)	Off by default	Doubles candidates; turn on per run
Multi-user / accounts	None in v1	Adding auth is the first step toward selling it